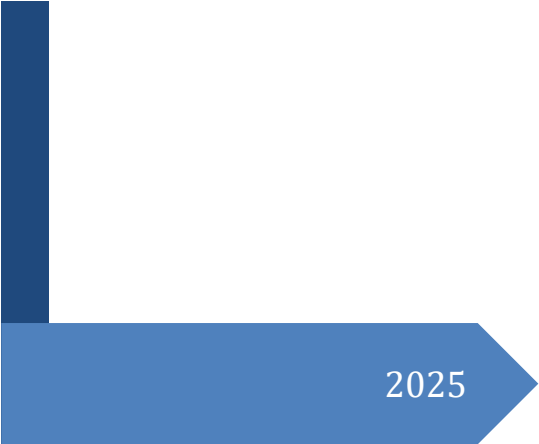




2025

GlobaChat

CSCI323 - Project

Advaith Sujith - 7837524

Asraa Shaikh - 7859557

Anushka Roy - 7753226

Rhythm Shahi - 7877304



Table of Contents

Acknowledgment	2
Executive Summary	3
Motivation	4
Technical Requirements	5
Main Constructs	5
Aspect 1: Motivation	5
Aspect 2: Solution Implementation	6
Aspect 3: Solution Evaluation and Lesson Learned	7
Implementation	7
Challenges & Limitations	13
Challenges	13
Limitations	13
Conclusion	14

Acknowledgment

We would like to express our sincere gratitude to **GlobaCom** and **Mr. Sashwat Dubey** for granting us the opportunity to work on this project. Their support, insights, and valuable guidance have been instrumental in shaping our approach and enhancing our understanding of AI-driven document analysis. This project has been a tremendous learning experience, and we truly appreciate their encouragement and trust in our work.

Additionally, we extend our heartfelt thanks to the **University of Wollongong in Dubai** and **Professor Patrick Mukala** for providing us with the platform and support to undertake this project. Their mentorship and encouragement have played a crucial role in our learning journey, enabling us to explore innovative solutions and refine our technical skills.

Executive Summary

This project addresses the widespread challenge of efficiently extracting and summarizing important information from the constantly increasing number of PDF documents. The capacity to swiftly and precisely extract crucial information from lengthy documents is necessary for well-informed decision-making and increased productivity in today's data-rich environment. To address this need, we created a unique platform enabling users to upload PDF files easily. These files are then automatically processed to produce thorough summaries and responses to a predetermined set of questions.

This platform can comprehend, interpret, and synthesize textual data with amazing accuracy due to the strength of Natural Language Processing (NLP) algorithms, which are at the core of this solution. Through the use of these cutting-edge AI algorithms, the platform simplifies the frequently time-consuming information retrieval process, enabling users to swiftly obtain the information they require without having to manually read through numerous pages of text. For organizations that handle a lot of paperwork, including finance, law, healthcare, and academics, where accuracy is crucial and time is of the essence, this capability offers substantial advantages.

This platform automatically responds to a series of pre-formulated queries, extracting specific information that is most relevant to the user's needs, in addition to summarizing the entire file and providing a summary of its contents. The platform's usefulness is further increased by this automatic question-answering feature, which lets users find important data points quickly without having to spend hours looking for them manually.

Motivation

The necessity to effectively extract essential information from the expanding volume of PDF documents is what motivates this effort. In today's fast-paced world, individuals and organizations across various industries must face the time-consuming challenge of searching through vast amounts of textual data to locate specific information. In addition to being time-consuming, this manual review method is prone to mistakes and inconsistencies, which reduces productivity and makes it more difficult to make well-informed decisions. By offering a cutting-edge AI-driven platform that automates the summarization process and intelligently extracts answers to predefined and new questions, this project directly addresses these issues and provides a game-changing solution that drastically reduces errors, saves time, and boosts overall productivity.

The strategic choice to use AI as the center of this solution derives from its unmatched capacity to digest enormous amounts of text quickly, recognize complex patterns, and extract relevant data with little assistance from humans. The platform can quickly uncover important insights and connections that would otherwise be hidden inside the papers because of AI algorithms' ability to evaluate and synthesize textual material at speeds significantly faster than humans. This application might be used, for example, to automatically go through a big collection of utility bills and determine which company is using the most electricity.

By automating these time-consuming tasks, which are prone to mistakes, this platform enables users to free themselves from the repetitiveness of manual document review and concentrate on more complex tasks and strategic making, which in turn stimulates innovation and increases success in their respective domains.

Technical Requirements

Programming Languages: Python, HTML, JavaScript

Libraries: Torch, Google Cloud, pymupdf, fitz, gc, re, transformers, pprint.

Hardware: T4 GPU, 15GB RAM, 12.7GB System RAM

Tools: Ngrok, deepseek R1.

Main Constructs

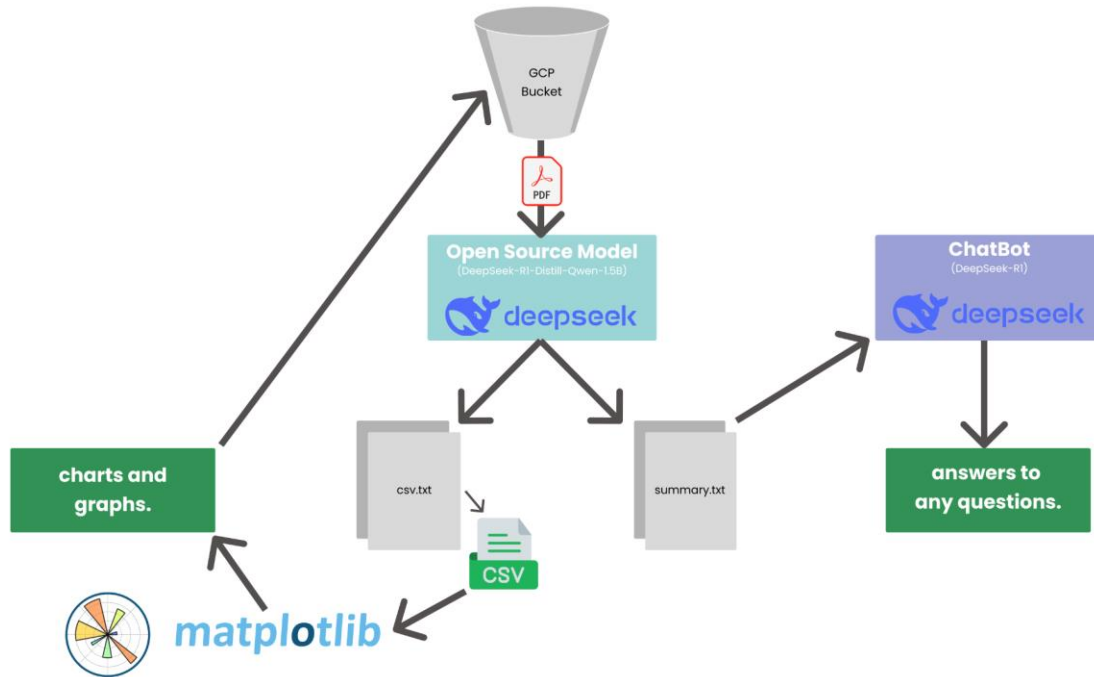
Aspect 1: Motivation

As mentioned earlier, organizations and individuals frequently handle large volumes of PDF documents, which often contain critical information that must be extracted efficiently. The main problem is the time-consuming process of manually sifting through these files to find key details. This challenge is particularly significant in environments where timely access to information is essential for decision-making and handling operations.

Mr. Shashwat Dubey, who is part of GlobaTech, identified this issue and sought assistance to build a solution that leverages AI to enhance the efficiency of information extraction and summarization from PDF files. We envisioned a platform powered by a Large Language Model (LLM) that could automate and streamline the information retrieval process.

By utilizing Natural Language Processing (NLP), AI can quickly analyze and summarize vast amounts of text, allowing users to access key information without manual searching. This will ensure higher accuracy in extracting relevant details based on predefined questions or new ones, minimizing errors associated with human processing.

Aspect 2: Solution Implementation



This is the architecture of our platform. It illustrates a data processing pipeline using an open-source AI model (DeepSeek-R1-Distill-Qwen-15B) that extracts information from PDFs stored in a Google Cloud Platform (GCP) bucket. The breakdown of data flow is as follows:

- Data Source (GCP Bucket):
 - The PDF documents are stored in a GCP Bucket and serve as the input.
- Processing by the Open-Source Model (DeepSeek-R1-Distill-Qwen-15B):
 - The AI model processes the PDF to extract information.
 - The extracted data is stored in two formats:
 - csv.txt (csv-formatted text file)
 - summary.txt (summarized version of the document)
- Data Utilization:
 - The csv.txt file is further processed into an actual CSV file.
 - Matplotlib is used to create charts and graphs from the data.
 - The charts and graphs are sent back to GCP Bucket for storage.
- Chatbot Integration (DeepSeek-R1):
 - The summarized file is sent to the chatbot.
 - The chatbot can then answer a set of questions, both predefined and new ones.

Aspect 3: Solution Evaluation and Lesson Learned

Interacting with Mr. Shaswat Dubey from GlobaTech provided us with valuable insights into the practical challenges organizations face when dealing with multiple PDF documents. Working with him made us realize the critical need for solutions that automate information extraction and ensure accuracy and efficiency in real-world scenarios. The learning experience made it clear how crucial it is to match AI solutions to particular business requirements as determined by those directly involved in the issue.

The practical experience we gained through this project and working with GlobaTech augmented the theoretical knowledge we acquired in our classes. Applying natural language processing techniques to automate the summarization process and answer predefined questions provided a deeper understanding of the capabilities and limitations of AI in real-world applications. This hands-on experience reinforced the importance of considering factors such as data quality and algorithm selection when developing AI-driven solutions for information management.

Our AI solution offers several potential benefits to GlobaTech. By automating the extraction and summarization of key information from PDF files, this platform can significantly reduce the time and resources required for manual document review. This efficiency gain can enable GlobaTech to improve decision-making, streamline operations, and enhance overall productivity. Furthermore, the platform's ability to accurately extract relevant details based on predefined questions can minimize errors associated with human processing, ensuring that critical information is readily available when needed.

Implementation

The code below handles the saving of the generated summaries and CSV data to separate output files, then we imported the necessary machine learning libraries and defined a function to manage and clear the memory used by the GPU and then we connected to Google Cloud Storage (GCS) to download files from GCS bucket to the local system and upload local files to GCS bucket.

```
with open(summary_output_filename, "w", encoding="utf-8") as summary_file:
    summary_file.write(summaries)
    print(f"Summaries saved to {summary_output_filename}")

with open(csv_output_filename, "w", encoding="utf-8") as csv_file:
    csv_file.write(csv_data)
    print(f"CSV data saved to {csv_output_filename}")
```

✓ 6m 7.6s

Python

```

import torch
import fitz
import gc
import re
from transformers import pipeline
import pprint

pdf_filename = "/content/231220241118460852reajan24.1d.pdf"
summary_output_filename = "summary_output.txt"
csv_output_filename = "csv_output.txt"

def clear_gpu_memory():
    torch.cuda.empty_cache()
    torch.cuda.ipc_collect()
    gc.collect()
    torch.cuda.reset_peak_memory_stats()

pipe = pipeline("text-generation", model="deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B", trust_remote_code=True)

summaries = ""
csv_data = ""

```

```

def download_file(bucket_name, file_name, destination_file_name, max_retries=3):
    bucket = storage_client.bucket(bucket_name)
    blob = bucket.blob(file_name)

    for attempt in range(max_retries):
        try:
            blob.download_to_filename(destination_file_name)
            print(f"Downloaded {file_name} to {destination_file_name}")
            return True
        except Exception as e:
            print(f"Attempt {attempt + 1}: Failed to download {file_name}: {e}")

    print(f"Failed to download {file_name} after {max_retries} attempts.")
    return False

```

[3] ✓ 0.0s

Python

```

def upload_to_gcp(bucket_name, source_file_name, destination_blob_name):
    bucket = storage_client.bucket(bucket_name)
    blob = bucket.blob(destination_blob_name)

    try:
        if not os.path.exists(source_file_name):
            print(f"File does not exist locally: {source_file_name}")
            return
        blob.upload_from_filename(source_file_name)
        print(f"Uploaded {source_file_name} to gs://{bucket_name}/{destination_blob_name}")
    except Exception as e:
        print(f"Failed to upload {source_file_name}: {e}")

```

[4] ✓ 0.0s

Python

The system authenticates with GCS using service account credentials to ensure secure access and management of files in the cloud. The code opens a PDF file and iterates through its pages in batches, extracting text used to generate summaries and CSV data through a pipeline. The generated outputs are formatted and then saved as text files locally. These files are uploaded to a designated GCS bucket, ensuring their storage and accessibility.

```

import os
from google.cloud import storage
from google.oauth2 import service_account

credentials_info = {
    "type": "service_account",
    "project_id": "supple-hangout-451519-m8",
    "private_key_id": "1168d80a3ca5a22c3077a1d12078eb5292237897",
    "private_key": "-----BEGIN PRIVATE KEY-----\nMIIEVwIBADANBgkqhkiG9w0BAQEFAASCCKwggSLAgEAAoIBAQQDyYhY2ZucrPCs7\N1Jt/17VHFaROLUdOpwbzPcOYpPFROMla7O1Y2xdT\n",
    "client_email": "retiapi@supple-hangout-451519-m8.iam.gserviceaccount.com",
    "client_id": "110106647507484432806",
    "auth_uri": "https://accounts.google.com/o/oauth2/auth",
    "token_uri": "https://oauth2.googleapis.com/token",
    "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
    "client_x509_cert_url": "https://www.googleapis.com/robot/v1/metadata/x509/retiapi%40supple-hangout-451519-m8.iam.gserviceaccount.com",
    "universe_domain": "googleapis.com"
}

credentials_obj = service_account.Credentials.from_service_account_info(credentials_info)

storage_client = storage.Client(credentials=credentials_obj, project=credentials_info['project_id'])

[2] ✓ 0.6s Python

with fitz.open(pdf_filename) as pdf_doc:
    num_pages = 7
    batch_size = 7

    for start_page in range(2, num_pages, batch_size):
        try:
            end_page = min(start_page + batch_size, num_pages)
            text = "\n".join([pdf_doc[page].get_text() for page in range(start_page, end_page)])

            summary_prompt = f"Briefly summarize this text:\n{text}"
            summary_messages = [{"role": "user", "content": summary_prompt}]
            summary_output = pipe(summary_messages, max_new_tokens=3000)
            raw_summary = [item["content"] for item in summary_output[0]["generated_text"] if item["role"] == "assistant"]
            cleaned_summary = "\n".join(raw_summary)
            cleaned_summary = re.split(r"</think>", cleaned_summary, maxsplit=1)[-1].strip()
            summaries += f"Summary {ctr}:\n(cleaned_summary)\n\n"

            csv_prompt = f"Extract any interesting data from the text and format it strictly as CSV (comma-separated values), without any additional text,"
            csv_messages = [{"role": "user", "content": csv_prompt}]
            csv_output = pipe(csv_messages, max_new_tokens=3000)
            raw_csv = [item["content"] for item in csv_output[0]["generated_text"] if item["role"] == "assistant"]
            cleaned_csv = "\n".join(raw_csv)
            cleaned_csv = re.split(r"</think>", cleaned_csv, maxsplit=1)[-1].strip()
            csv_data += f"CSV Data {ctr}:\n(cleaned_csv)\n\n"

            pprint.pprint(cleaned_summary)
            pprint.pprint(cleaned_csv)
            print(f"Processed pages {start_page+1} to {end_page}, summary and CSV data generated.")

        except Exception as e:
            print(f"Error processing pages {start_page+1} to {end_page}: {e}")

    del summary_output, raw_summary, cleaned_summary
    del csv_output, raw_csv, cleaned_csv
    del summary_messages, summary_prompt, csv_messages, csv_prompt, text
    clear_gpu_memory()

    upload_to_gcp("retinopathy_images", "csv_output.txt", "csv_output.txt")
    upload_to_gcp("retinopathy_images", "summary_output.txt", "summary_output.txt")

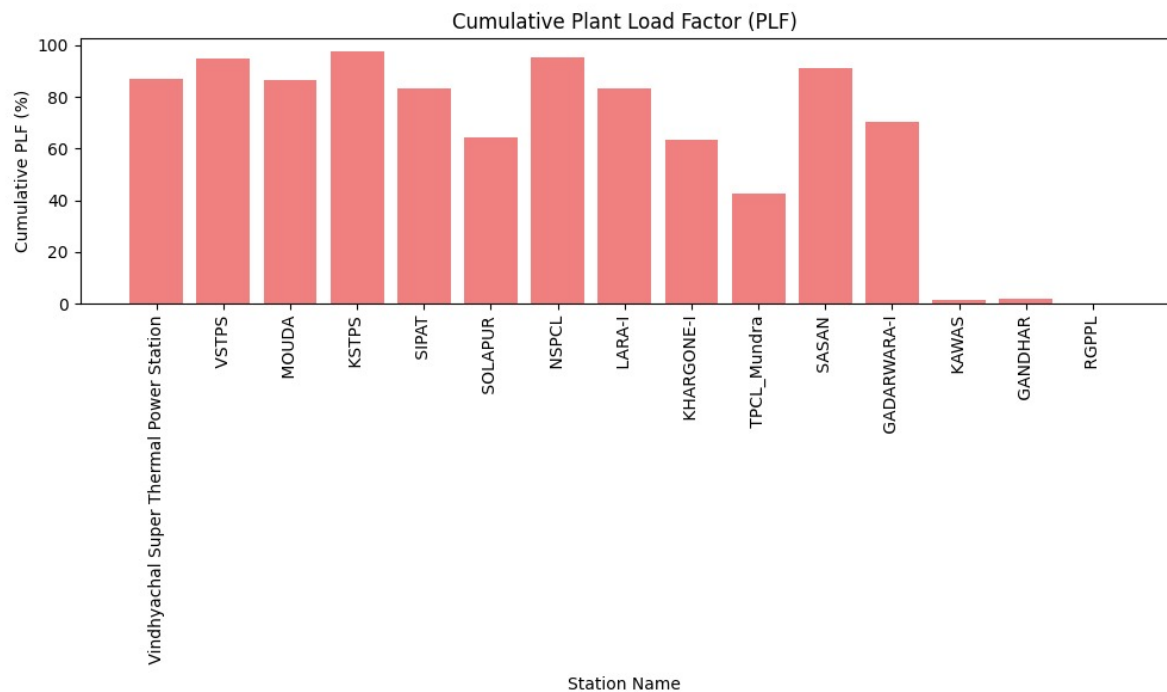
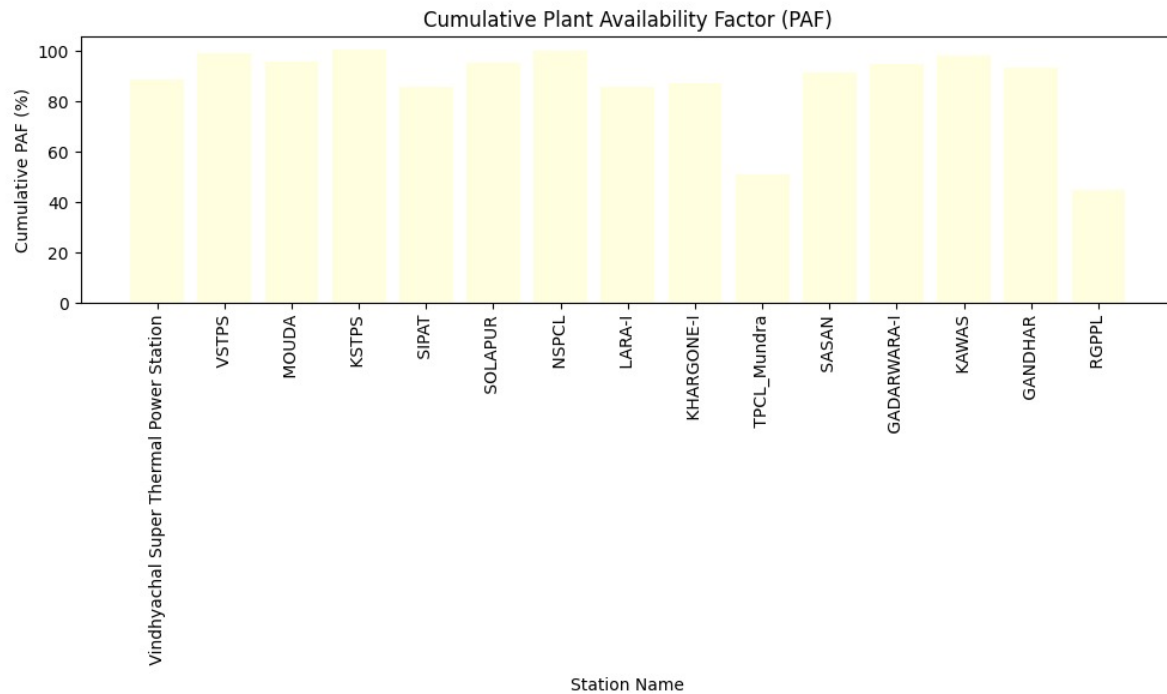
[9] ✓ 1.6s Python

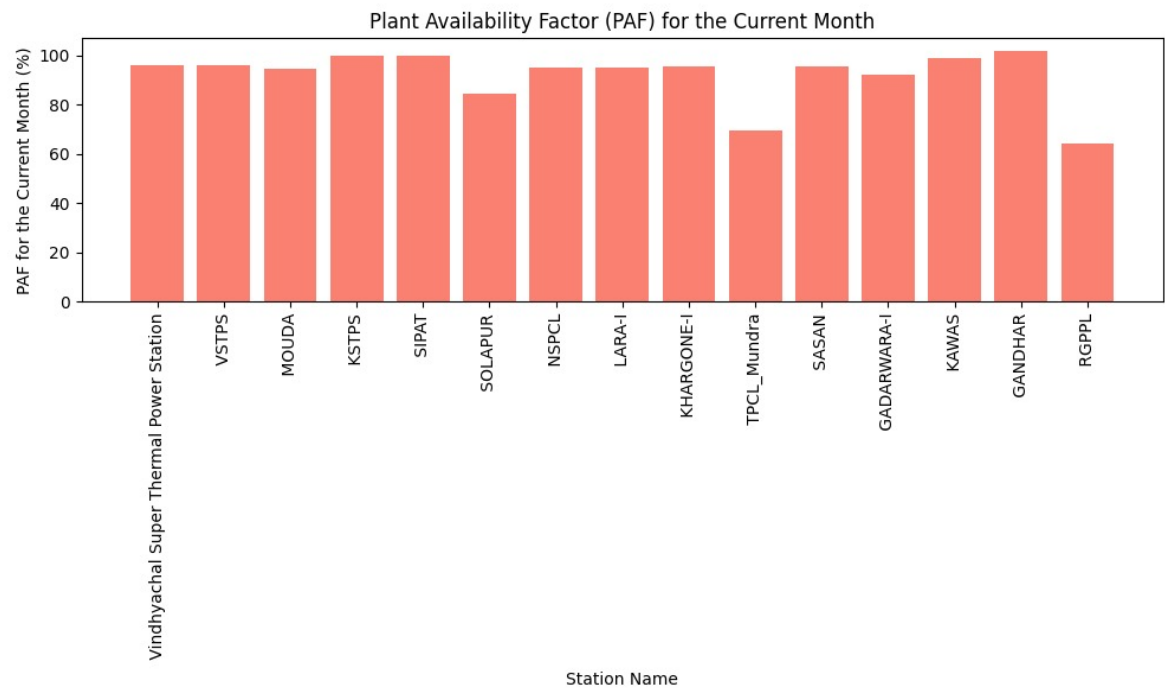
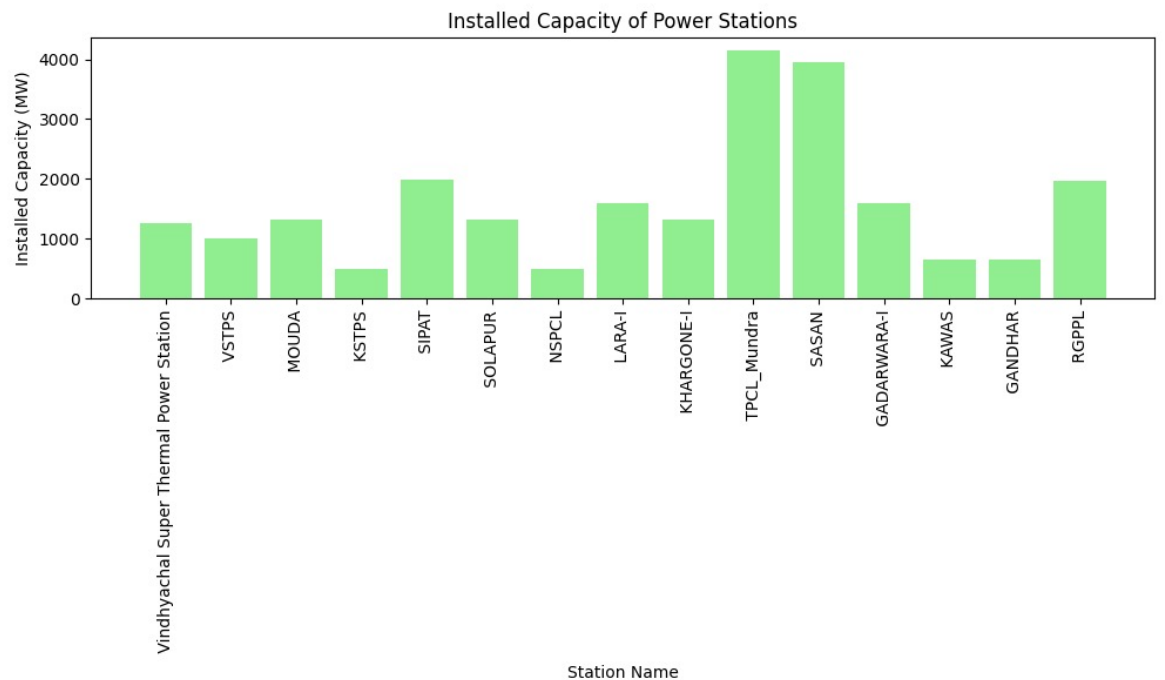
... Uploaded csv_output.txt to gs://retinopathy_images/csv_output.txt
Uploaded summary_output.txt to gs://retinopathy_images/summary_output.txt

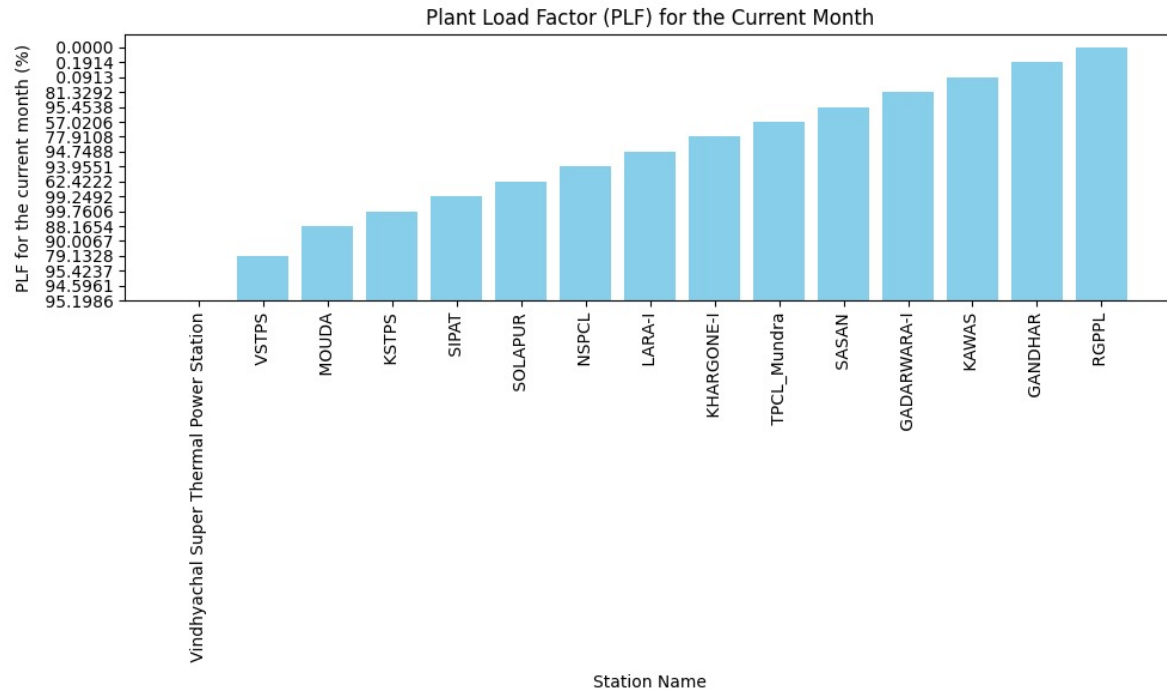
```

☐	 csv_output.txt	264 B	text/plain
☐	 summary_output.txt	2.7 KB	text/plain

The following graphs are generated using our model and Matplotlib. Since each month's PDF contains similar data, we can consistently produce graphs like these for every PDF.







This is our chatbot interface, GlobaChat. This enables users to engage with the PDF files in a dynamic way, asking questions and receiving responses.

GlobaChat

tell me a bit about the pdf

Ask!

The PDF outlines a methodology to assess a plant's contribution to a company's financial performance during high demand seasons, focusing on both operational metrics and financial data. Here's a concise overview:

Key Sections of the PDF:

1. **Plant Performance Metrics:**
 - Analyzes data like **Plant Load Factor (PLF)**, cumulative high/low demand season performance, and energy generation capacity.
 - Emphasizes seasonal operation (e.g., High Demand Season vs. Low Demand Season) and how these periods impact the plant's output.
2. **Financial Analysis (REA Account Data):**
 - Reviews financial indicators such as **net profit**, **operating profit/loss**, and seasonal profit breakdowns.
 - Links plant performance to revenue generation, particularly during peak demand periods.
3. **Methodology:**
 - **Step-by-Step Process:** Compares plant operational efficiency (e.g., capacity utilization)

Challenges & Limitations

Challenges

The primary constraint we encountered during the project was the lack of adequate computational resources. We relied on an open-source model, which demanded significant GPU power and system RAM. The available resources were insufficient, which led to the necessity of using Google Colab's resources. However, even with Google Colab's resources, there was still not enough GPU and RAM to support our project's needs. This constraint significantly impacted our project. This led to us choosing a smaller model. While efficient in terms of resource utilization, it exhibited reduced accuracy compared to other full-fledged models.

Another significant challenge was the dependency on the quality and consistency of the PDF documents before getting processed and while being processed. The documents were poorly formatted and contained a lot of unwanted information. Without formatting these documents, the accuracy and efficiency of the AI algorithms would have been compromised.

Limitations

Due to these constraints, our AI model has several limitations. One of the primary concerns is the size of the PDF file, as excessively large files may exceed the memory limits of the Google Colab GPU, making it difficult to process them efficiently. This means that users may need to break down larger documents into smaller sections for analysis.

Another significant limitation is the extended processing time required for analyzing and summarizing PDF files. This is a direct consequence of the computational constraints, which restrict the size and complexity of the AI model that can be utilized. As a result, while the model is designed to provide accurate and relevant insights, users may experience delays depending on the length and complexity of the documents being processed.

Finally, our chatbot also has a limit on the number of characters it can process at one time. This means that particularly long text inputs may need to be truncated or split into smaller segments for analysis. While this restriction ensures stable performance, it can sometimes require additional effort from the user to format inputs appropriately for the best results.

Conclusion

Through this project, we have successfully developed and deployed an LLM-powered solution for efficient PDF summarization and information extraction. By automating the time-consuming task of going through PDFs, this model offers significant benefits in terms of productivity, accuracy, and decision-making. The collaboration with Globatech also provided valuable real-world insights and demonstrated the model's potential to address critical business needs in such industries.

The utilization of the DeepSeek open-source AI model has also been crucial in enabling these functionalities, showcasing the transforming potential of AI technology in addressing GlobaTech's specific document processing needs

Utilizing better computational resources in the future could enable the development of a full-fledged platform capable of handling a wider range of tasks and document types. We are confident that this LLM platform will have a lasting impact on how GlobaTech efficiently extracts and utilizes information from their growing volume of PDF documents, ultimately contributing to their operational efficiency and strategic decision-making.